Break the helper: answer key

Compare it with your sheet. Each TRY IT box is code you can paste into the notebook to see the answer for yourself. The outputs shown are what you should get.

How to brainstorm ways to break a function

When you're handed a helper function, go through four categories of input that might break it:

Values	Shape	Type	Missing / caller
everyone gives the same answer · one extreme answer	one respondent · one question · square (as many respondents as questions) · empty	a list · integers · yes/no (True/False) · text like `'7'`	a skipped question (`nan`) · does it change my data?

For each idea, ask three questions: **What happens? Would I notice? What should happen?**

> **Example, "one respondent":** What if only one person has taken the survey so far? Then each question has one answer. Its mean is that answer and its spread is 0, so line 4 divides by 0. Would I notice? Run it and find out (see Part 3).

Part 1: trace it by hand

Step	Answer	Why
(a) `mu_vec`	`[2., 7.]`, shape `(2,)`	One mean per question. Q1: (1+3+1+3)/4 = 2. Q2: everyone said 7.
(b) `sigma_vec`	`[1., 0.]`, shape `(2,)`	Everyone gave the same answer to Q2, so its spread is **0**.
(c) `data - mu_vec`	`[[-1, 0], [1, 0], [-1, 0], [1, 0]]`	`(4, 2) - (2,)` broadcasts down the rows: each respondent minus the question means.
(d) the value returned	Q1: `[-1, 1, -1, 1]`. Q2: **`nan`** for every respondent.	Q2 is 0 / 0, which is undefined.
(e) What goes wrong?	The function divides by 0, but there is **no error**. Python prints a warning, keeps going and returns `nan` ("not a number").	

TRY IT Notice two things: it's a **warning, not an error**, and the result has **nan** in it.

```
d = np.array([[1., 7.], [3., 7.], [1., 7.], [3., 7.]])
standardize_features(d)

RuntimeWarning: invalid value encountered in divide
array([[-1., nan],
       [ 1., nan],
       [-1., nan],
       [ 1., nan]])
```

Why it matters: `nan` spreads. Any later calculation that uses it is also `nan`; for example, `np.mean([1, np.nan])` is `nan`. If you averaged each respondent's standardized answers, every respondent's score would be `nan`.

Part 2: shapes and broadcasting

Row	data.shape	axis	mu_vec.shape	Runs?	Correct?
B	(200, 5)	1	(200,): one mean per respondent	**Error**: compare 5 vs 200	—
C	(5, 5)	0	(5,)	Yes	Yes (this is the correct function)
D	(5, 5)	1	(5,): one mean per respondent	Yes: 5 vs 5 match	**No**

(f) D. It runs and returns wrong numbers with no error and no warning. Row B has the same bug, but it crashes right away, so you'd find it. With a pilot survey that happens to be square, you'd never know. **Lesson:** test with data where the number of respondents ≠ the number of questions.

TRY IT **Row B: the bug crashes on the full survey, which is the good outcome.**

```
def standardize_axis1(data):          # the classmate's version
    mu_vec = data.mean(axis=1)
    sigma_vec = data.std(axis=1)
    return (data - mu_vec) / sigma_vec

standardize_axis1(survey)             # the 200 x 5 survey from the notebook
ValueError: operands could not be broadcast together with shapes (200,5) (200,)
```

TRY IT **Row D: the same bug on square data runs and gives a wrong answer.** Here's a 2 × 2 survey (2 respondents, 2 questions); a 5 × 5 works the same way.

```
#             Q1  Q2
d = np.array([[1., 4.],    # respondent 0
              [3., 8.]])   # respondent 1
```

What we want (axis=0, by question):

```
mu_vec    = d.mean(axis=0)     # [2. 6.]   one mean per QUESTION
sigma_vec = d.std(axis=0)      # [1. 2.]
standardize_features(d)        # [[-1. -1.]
                               #  [ 1.  1.]]   each question: mean 0, std 1  ✓
```

What the bug does (axis=1):

```
mu_vec    = d.mean(axis=1)     # [2.5 5.5]  one mean per RESPONDENT
sigma_vec = d.std(axis=1)      # [1.5 2.5]

d - mu_vec                     # shapes (2, 2) - (2,): match, so it broadcasts...
# [[1 - 2.5,  4 - 5.5],        # ...but [2.5, 5.5] is laid across each ROW:
#  [3 - 2.5,  8 - 5.5]]        # respondent 1's mean is subtracted from Q2's column
# = [[-1.5, -1.5],
#    [ 0.5,  2.5]]

standardize_axis1(d)           # [[-1.    -0.6  ]
                               #  [ 0.333  1.   ]]
standardize_axis1(d).mean(axis=0)    # [-0.333  0.2  ]   not 0  ✗
```

Picture it: 2.5 is respondent 0's mean and 5.5 is respondent 1's. Broadcasting doesn't know that; it lays `[2.5, 5.5]` across the questions. So respondent 0's Q2 answer (4) has respondent 1's mean (5.5) subtracted from it. The result still looks like real z-scores.

TRY IT **How to make respondent means line up with respondents:** they need shape `(2, 1)`, not `(2,)`:

```
d.mean(axis=1, keepdims=True)    # [[2.5]
                                 #  [5.5]]   shape (2, 1): one per respondent, as a column
# (2, 2) - (2, 1) subtracts each respondent's own average: useful, but a different task.
```

Part 3: other ways it could break

There are many good answers. Here are nine, each with what actually happens. "Works fine" is a correct answer when it's true. The last column is what the function *should* do; there's more than one reasonable choice, and what matters is that you decide.

The survey	What actually happens	Circle	What it should do
Only one respondent `np.array([[3., 8., 5.]])`	`array([[nan, nan, nan]])` + RuntimeWarning. Every question's std is 0.	warning / wrong	Raise `ValueError("need at least 2 respondents")`
Someone skipped a question `np.array([[1., np.nan], [3., 4.], [5., 6.]])`	Q1 is fine: `[-1.225, 0, 1.225]`. Q2 is **all** `nan`, with **no warning at all**.	wrong answer	Ignore skipped answers (`np.nanmean`, `np.nanstd`) or raise an error
Answers in a list `[[1, 2], [3, 4]]`	`AttributeError: 'list' object has no attribute 'mean'`	error	Convert first: `np.asarray(data, dtype=float)`
Answers typed as text `np.array([['7', '8'], ['5', '6']])`	`.mean` raises a `TypeError`: these are strings, not numbers	error	Convert with `.astype(float)`, or raise a clear error
No respondents yet `np.empty((0, 3))`	Returns an empty array, plus several warnings ("Mean of empty slice")	warning	Raise `ValueError`
One extreme answer nine people answer 9 or 10, one answers 1	Runs. The 1 gets z = −2.96. But the 9s get 0.08 and the 10s get 0.46: the one extreme answer squishes everyone else together.	works / arguably wrong	Document it, or use robust scaling (median / IQR)
One question as a 1-D array `np.array([4., 6., 8.])`	Works: `[-1.225, 0, 1.225]`	works fine	Fine, or require 2-D
Integer answers (most surveys) `np.array([[2, 9], [6, 5], [4, 7]])`	Works: the result is float, `[[-1.225, 1.225], [1.225, -1.225], [0, 0]]`	works fine	Nothing to fix
A yes/no question (True/False) `np.array([[True, False], [False, False], [True, True], [True, False]])`	Works: True counts as 1 and False as 0. But if **everyone** says yes, it's a constant column: Part 1's `nan` again.	works fine	Fine; the constant-column fix covers "everyone said yes"

TRY IT **The skipped question is the sneakiest.** It's worse than Part 1, because this time there isn't even a warning:

```
x = np.array([[1., np.nan],
              [3., 4.],
              [5., 6.]])
standardize_features(x)
array([[-1.225,    nan],
       [ 0.   ,    nan],
       [ 1.225,    nan]])          # no error, no warning: one skipped answer ruined the
whole question
```

Part 4: what a good test should catch

Any four of these, with a sensible survey and expected result, are good answers. The first four are the most important, because each one catches a bug from this sheet.

#	A good test should catch...	Survey you'd test with	What a correct function returns
1	**Wrong numbers** (bad formula, forgot to divide by sigma)	A small survey you can work by hand: `[[1., 4.], [3., 8.]]`	`[[-1., -1.], [1., 1.]]`; in general, every question has mean 0 and std 1 (`np.allclose`)
2	**Everyone gives the same answer** (Part 1)	`[[1., 5.], [2., 5.], [3., 5.]]`	No `nan`: that question comes back as all 0 (or a clear error, if that's what you decided)
3	**A respondent/question mix-up** that square data hides (Part 2, row D)	A square survey, e.g. `[[1., 4.], [3., 8.]]`, **and** one with more respondents than questions	Each question has mean 0 and std 1, for both shapes
4	**A changed shape**	The full survey, `(200, 5)`	The same shape: `(200, 5)`
5	**A skipped question**	`[[1., nan], [3., 4.], [5., 6.]]`	The answers that aren't missing still get z-scores (or a clear error)
6	**A yes/no question**	`[[True, False], [False, False], [True, True], [True, False]]`	Works: means 0, stds 1, and every "yes" ends up above 0
7	**The wrong input type**	A list, or text answers like `'7'`	Same answer as the float array, or a clear error
8	**Changing the caller's data**	Save a copy of the survey, run the function, compare	The survey is unchanged afterward

TRY IT **From the list to code.** The notebook turns this list into three tests, from incomplete to complete, and runs each one on a correct version and three buggy ones:

Test	What it checks (list items)	What it misses
`test_1_runs` incomplete	Only that it runs on the survey and keeps the shape (4)	Passes on `forgot_sigma`, which is wrong: it never looks at the numbers
`test_2_properties` better	Every question has mean 0 and std 1 on the 200 × 5 survey (1, 4)	Passes on the original, which breaks when everyone gives the same answer: the survey never has that
`test_3_edge_cases` complete	A square survey with a hand-worked answer, a question everyone answered the same, a yes/no question (1, 2, 3, 6)	Catches every buggy version in the notebook

Common mistakes when writing tests:

Mistake	Example	Why it's a problem
Comparing floats with `==`	`assert z.mean(axis=0) == 0`	Fails on rounding (`3e-17`), and an array in an `assert` raises an error
Testing only data that already works	`d = survey` (the 200 × 5)	Passes even with the Part 1 bug, so it catches nothing new
An assert that checks nothing	`assert z` or `assert f(d) is not None`	Passes no matter what the numbers are
Testing the bug, not the goal	`assert np.isnan(z).any()`	That passes *because* of the bug. Ask yourself: what should happen?

Part 5

> Any version of: *"The code can run and still return nan or wrong numbers. A warning isn't an error, and broadcasting can match shapes by accident. Only a check against what you expect catches that."*

Next: see it in the notebook

Open the section **"Testing the helper: checking the Break the helper worksheet"** in `M5_numpy_in_class.ipynb`. It follows this sheet part by part with a 200-respondent survey: run Parts 1–3 to check your answers, then compare your Part 4 list with the three tests and the summary table at the end.